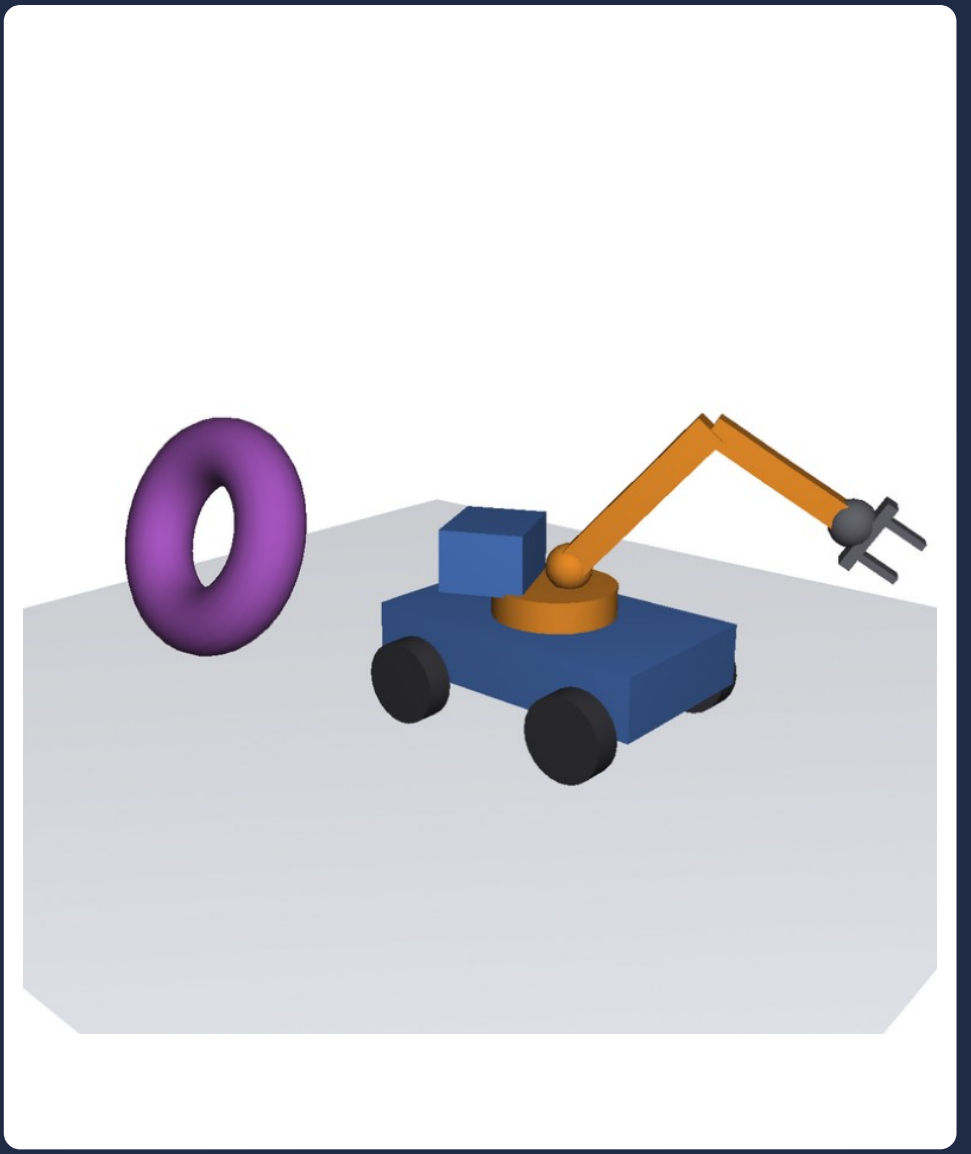
INFORMÁTICA GRÁFICA Y VISUALIZACIÓN · PRÁCTICA 2

Práctica 2 · Representación de modelos 3D e interacción

Lo que hay que resolver

Una malla paramétrica propia, un modelo articulado con grafo de escena, selección con el ratón, la Práctica 1 integrada y animación automática



Prof. Juan José Jiménez Delgado
Grado en Ingeniería Informática · Universidad de Jaén
Curso 2026-27

PRESENTACIÓN

La práctica en cifras



5

sesiones de laboratorio (10 h)

4,0

puntos sobre 10 (ponderado al 50 % en prácticas)

4 · 12

niveles y nodos, como mínimo, en el grafo de escena

4

grados de libertad, como mínimo, en el modelo articulado

Contenidos

- Mallas de triángulos mediante arrays de vértices (y de normales).
- Grafo de escena y pila de matrices.
- Interacción: teclado, ratón y menús.

Cómo se trabaja

- En parejas, a partir del código de los ejemplos explicativos **pr2a, pr2b y pr2c** (ya resueltos) y de la Práctica 1.
- En la última sesión hay un **examen práctico**.

Qué hay que hacer, de un vistazo



1 Objeto paramétrico

Una malla propia con arrays de vértices y de normales, distinta del cilindro; plano o interpolado desde el menú.

2 Modelo articulado

Grafo de escena con primitivas: ≥ 4 niveles, ≥ 12 nodos, ≥ 4 grados de libertad, no lineal y con límites.

3 Selección con ratón

Buffer de color: clic en una parte y arrastrar para moverla. Modo teclado o ratón desde el menú.

4 Práctica 1 integrada

Transformaciones sobre los dos modelos, movimientos de cámara y viewports.

5 Animación

Animación automática del modelo y órbita automática de la cámara, activadas por menú.

6 Experiencia de usuario

Menús, teclado y ratón bien repartidos: es un aspecto que se valora.

Contenidos

1 **El objeto paramétrico**
Parametrización, arrays de vértices y normales, plano o interpolado

2 **El modelo articulado**
Requisitos, grafo de escena, pila de matrices, grados de libertad y límites

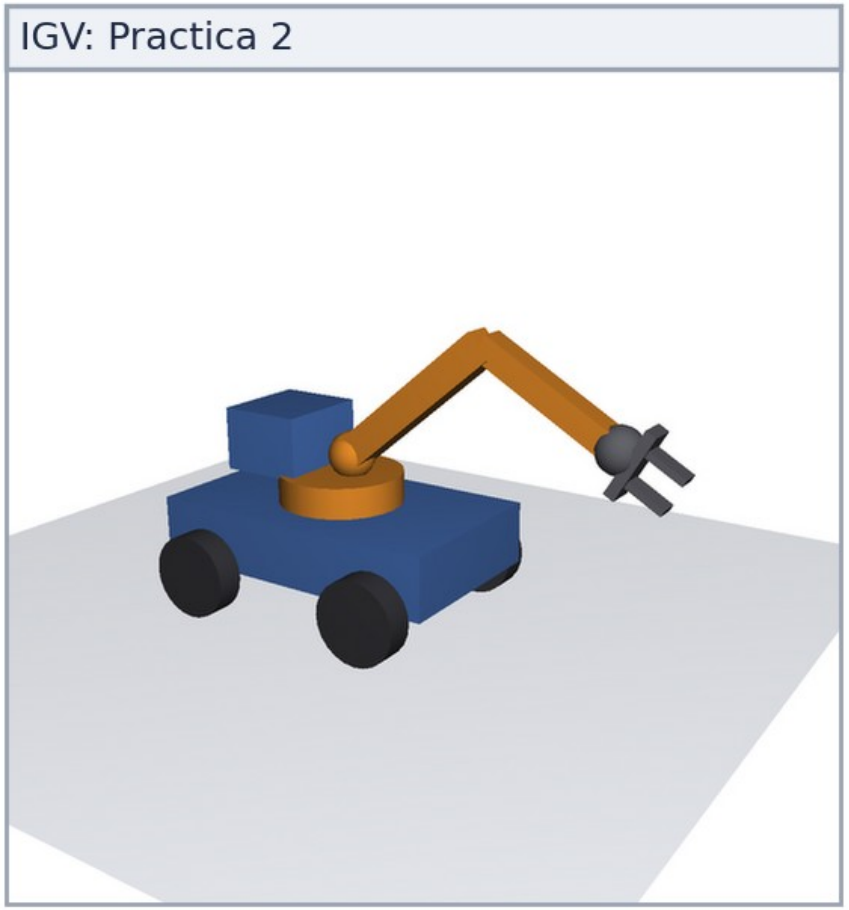
3 **Selección con el ratón**
Buffer de color, resaltado, arrastre y cambio de modo

4 **Integración de la Práctica 1**
Transformaciones, cámara y viewports con los dos modelos

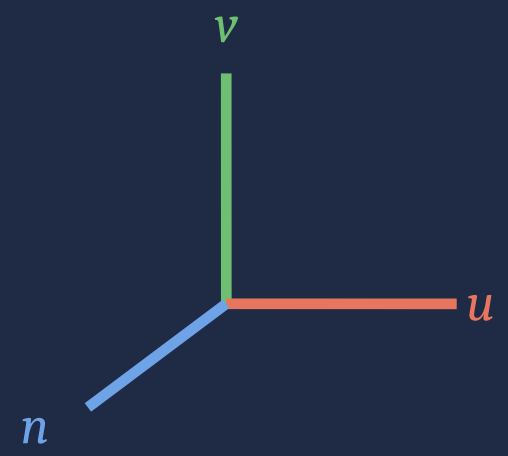
5 **Animación automática**
idleFunc, tiempo real, límites y órbita de la cámara

6 **Experiencia de usuario**
Menús, mapa de teclas y ayuda

7 **Entrega**
Qué entregar y cómo comprobar que está todo



1



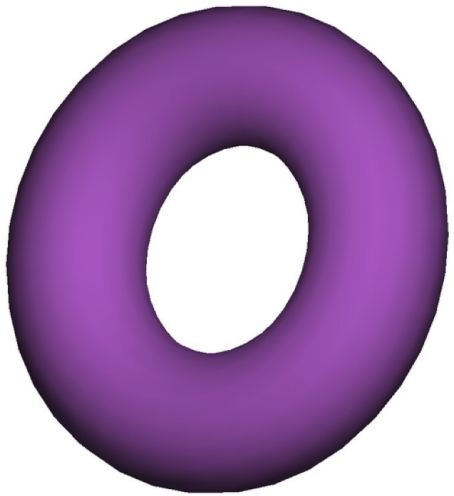
El objeto paramétrico

Una malla de triángulos propia, con arrays de vértices y de normales de OpenGL.

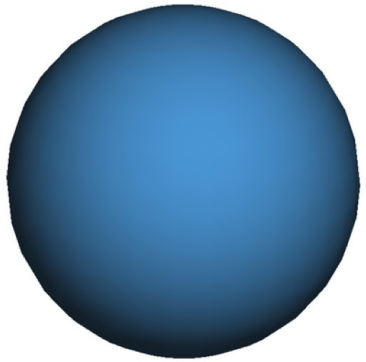
Qué se pide



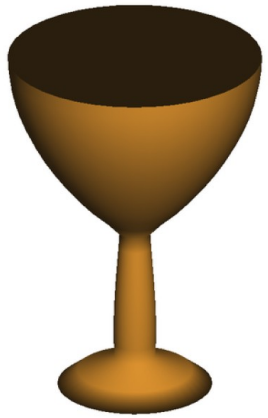
- Un objeto modelado con **igvMallaTriangulos** y generado **paramétricamente**, como el cilindro de pr2a: esfera, toro, hexaedro, superficie de revolución… **no muy parecido al cilindro**.
- Vértices en el **array de vértices** de OpenGL y un **array de normales** (si no hay fórmula para ellas, se calculan a partir de los triángulos).
- Visualización con **sombreado plano** e **interpolado**, elegible desde una **opción de menú**.



Toro: cerrado en u y en v



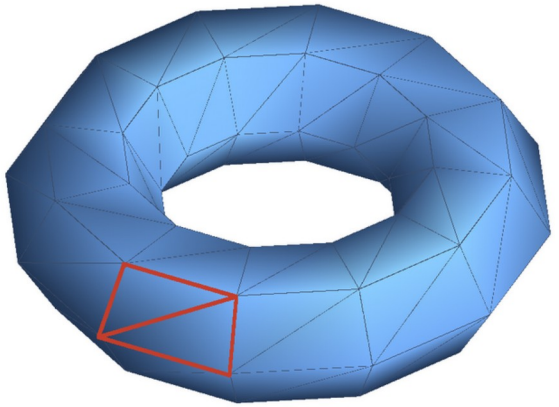
Esfera: polos en $v = 0$ y $v = \pi$



Copa: revolución de un perfil

De la parametrización a la malla





$$x = (R + r\cos v)\cos u$$
$$y = r\sin v$$
$$z = (R + r\cos v)\sin u$$

$u, v \in [0, 2\pi)$

Normal: $(\cos v \cos u, \sin v, \cos v \sin u)$

- Vértice $(i, j) \rightarrow$ índice $k = j \cdot \text{divU} + i$.
- num_vertices = $\text{divU} \cdot \text{divV}$ y num_triangulos = $2 \cdot \text{divU} \cdot \text{divV}$.
- Cada cuadrilátero abcd da los triángulos (a, d, b) y (b, d, c): antihorarios vistos desde fuera.

Ejemplo: el constructor de igvToro

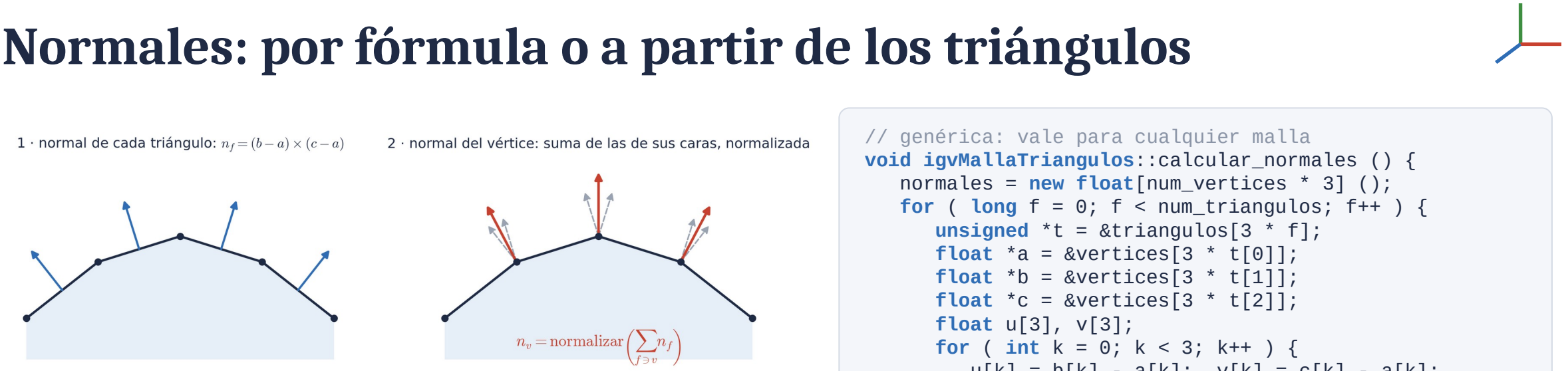


```
igvToro::igvToro ( float R, float r, int divU, int divV ) {
   num_vertices   = divU * divV;
   num_triangulos = 2 * divU * divV;
   vertices   = new float[num_vertices * 3];
   normales   = new float[num_vertices * 3];
   triangulos = new unsigned int[num_triangulos * 3];
   for ( int j = 0; j < divV; j++ ) {      // v: sección
      float v = 2 * M_PI * j / divV;
      for ( int i = 0; i < divU; i++ ) {   // u: vuelta
         float u = 2 * M_PI * i / divU;
         int k = ( j * divU + i ) * 3;
         vertices[k]   = ( R + r * cos(v) ) * cos(u);
         vertices[k+1] =   r * sin(v);
         vertices[k+2] = ( R + r * cos(v) ) * sin(u);
         normales[k]   = cos(v) * cos(u);
         normales[k+1] = sin(v);
         normales[k+2] = cos(v) * sin(u);
      }
   }
```

```
   int t = 0;                        // triángulos
   for ( int j = 0; j < divV; j++ )
      for ( int i = 0; i < divU; i++ ) {
         unsigned a = j * divU + i;
         unsigned b = j * divU + (i + 1) % divU;
         unsigned c = ((j + 1) % divV) * divU
                      + (i + 1) % divU;
         unsigned d = ((j + 1) % divV) * divU + i;
         triangulos[t++] = a;
         triangulos[t++] = d;
         triangulos[t++] = b;
         triangulos[t++] = b;
         triangulos[t++] = d;
         triangulos[t++] = c;
      }
}
```

igvToro hereda de **igvMallaTriangulos**, como igvCilindro: solo cambia el constructor. visualizar() (glVertexPointer, glNormalPointer y glDrawElements) **no se toca**.

Normales: por fórmula o a partir de los triángulos



Por fórmula

Si la superficie es paramétrica, la normal sale de sus derivadas (toro, esfera): exacta y sin coste.

A partir de los triángulos

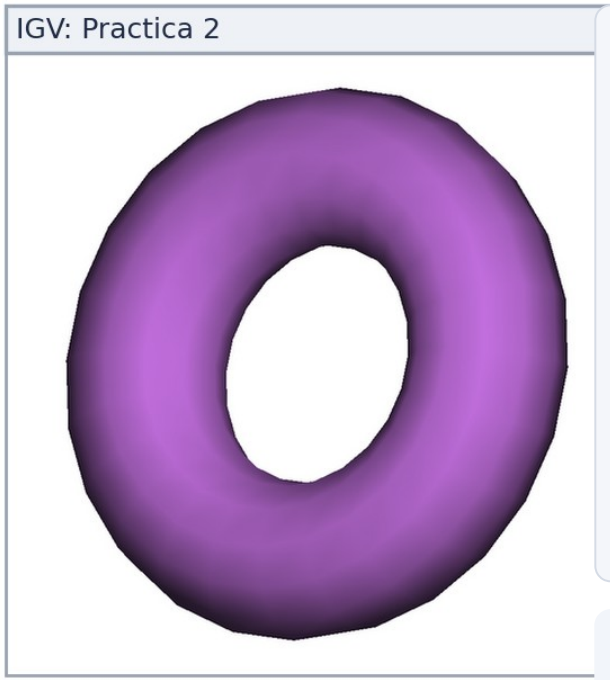
Para cualquier malla: sumar la normal de cada cara en sus tres vértices y normalizar.

```
// genérica: vale para cualquier malla
void igvMallaTriangulos::calcular_normales () {
   normales = new float[num_vertices * 3] ();
   for ( long f = 0; f < num_triangulos; f++ ) {
      unsigned *t = &triangulos[3 * f];
      float *a = &vertices[3 * t[0]];
      float *b = &vertices[3 * t[1]];
      float *c = &vertices[3 * t[2]];
      float u[3], v[3];
      for ( int k = 0; k < 3; k++ ) {
         u[k] = b[k] - a[k];  v[k] = c[k] - a[k];
      }
      float n[3] = { u[1]*v[2] - u[2]*v[1],
                     u[2]*v[0] - u[0]*v[2],
                     u[0]*v[1] - u[1]*v[0] };
      for ( int s = 0; s < 3; s++ )     // suma en
         for ( int k = 0; k < 3; k++ )  // sus vértices
            normales[3 * t[s] + k] += n[k];
   }
}  // GL_NORMALIZE (en visualizar) normaliza
```

Plano o interpolado, desde el menú



GL_FLAT: un color por triángulo



GL_SMOOTH: Gouraud

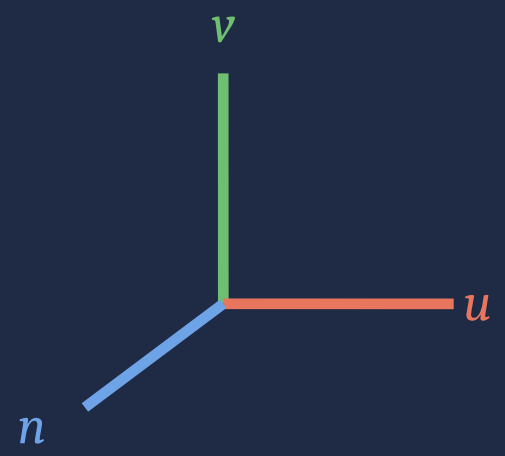
```
// en igvInterfaz::menuHandle ( int valor )
switch ( valor ) {
   case M_PLANO:
      malla->setGouraud ( false ); break;
   case M_GOURAUD:
      malla->setGouraud ( true );  break;
   // … resto de opciones
}
glutPostRedisplay ();
```

En pr2a se alternaba con la tecla g (cambiarvis()); aquí se pide **una opción de menú**.

Con las mismas normales

- Las dos imágenes usan el **mismo** array de normales: solo cambia glShadeModel.
- En GL_FLAT, OpenGL toma el color del **último vértice** de cada triángulo.

2

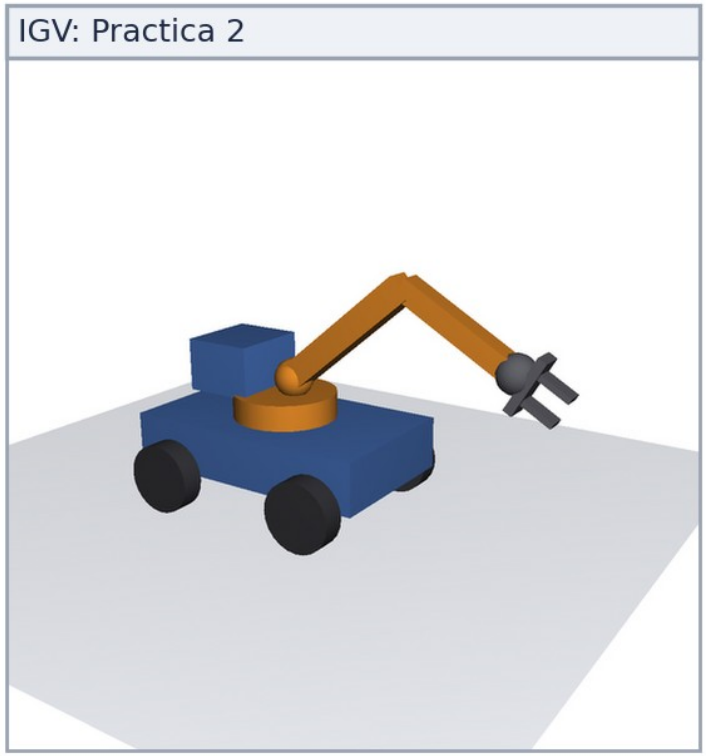


El modelo articulado

Un grafo de escena propio, con primitivas de OpenGL y la pila de matrices.

Qué se pide

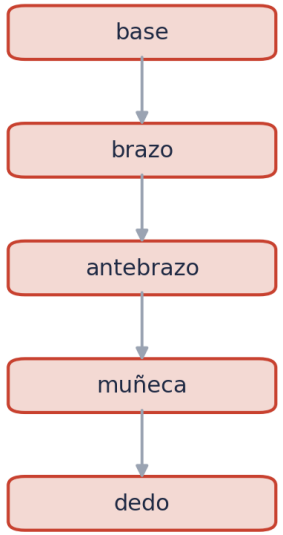
Requisito	Mínimo	En el robot de ejemplo
Niveles del árbol	4	6 (robot, torreta, brazo, antebrazo, pinza, dedos)
Nodos	12	25 (11 de transformación y 14 primitivas)
Grados de libertad	4, con rotaciones y alguna traslación	5: avance y pinza (traslaciones); giro, brazo y codo (rotaciones)
Estructura	no lineal	chasis, ruedas, cabina y dos dedos son ramas
Realismo	posiciones imposibles, no	cada grado de libertad tiene sus límites
Geometría	primitivas de OpenGL	cubos, cilindros y esferas



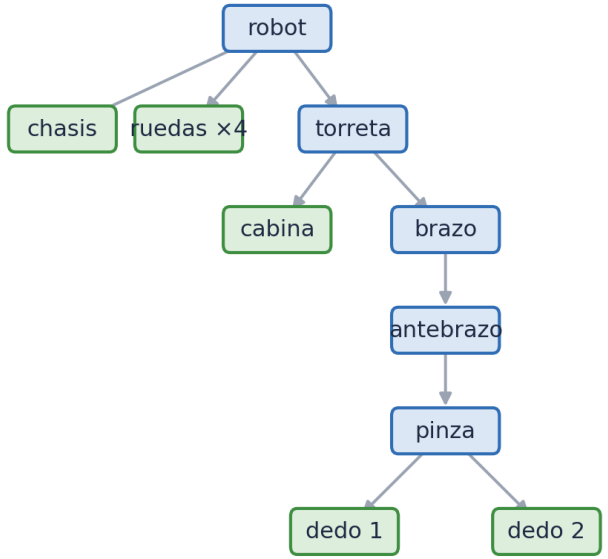
El diseño es **libre**, pero **no** puede ser el ejemplo del guion de pr2b. Se valora su realismo.

No lineal: un árbol, no una cadena

Lineal (cadena): NO vale



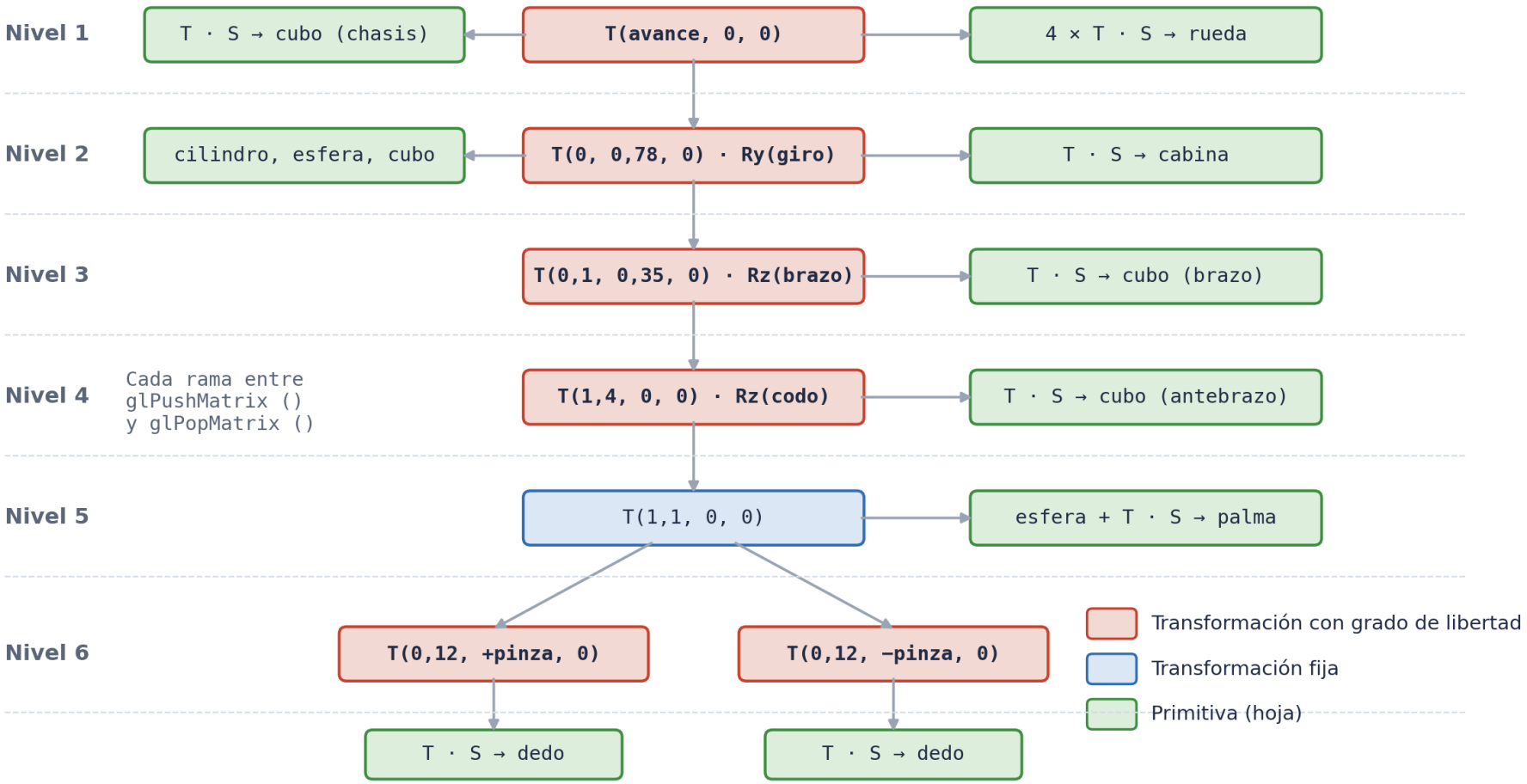
No lineal (árbol): cada nodo puede tener varios hijos



- En una **cadena**, cada nodo tiene un solo hijo: brazo → antebrazo → muñeca… No cumple el requisito.
- En un **árbol**, un nodo tiene varios hijos que heredan su transformación pero **no** se afectan entre sí.
- En código: cada hijo entre **glPushMatrix** y **glPopMatrix**.

En el ejemplo, el nodo **robot** tiene como hijos el chasis, las cuatro ruedas y la torreta; la **pinza** tiene dos dedos que se abren en sentidos opuestos.

El grafo de escena del ejemplo



Cada nodo hereda la matriz de sus antecesores. Las transformaciones **rojas** llevan los grados de libertad: son las que hay que señalar en la memoria.

visualizar() con la pila de matrices



```
void igvRobot::visualizar () {
   glPushMatrix ();
   glTranslatef ( avance, 0, 0 );          // nivel 1
   pintar_chasis ();  pintar_ruedas ();
   glTranslatef ( 0, 0.78, 0 );
   glRotatef ( giro, 0, 1, 0 );            // nivel 2
   pintar_torreta ();  pintar_cabina ();
   glTranslatef ( 0.1, 0.35, 0 );
   glRotatef ( brazo, 0, 0, 1 );           // nivel 3
   pintar_brazo ();
   glTranslatef ( 1.4, 0, 0 );
   glRotatef ( codo, 0, 0, 1 );            // nivel 4
   pintar_antebrazo ();
   glTranslatef ( 1.1, 0, 0 );             // nivel 5
   pintar_munieca ();
   pintar_dedo ( +pinza );                 // nivel 6
   pintar_dedo ( -pinza );
   glPopMatrix ();
}
```

```
void igvRobot::pintar_dedo ( float d ) {
   glPushMatrix ();
   glTranslatef ( 0.12, d, 0 );  // GDL
   glTranslatef ( 0.17, 0, 0 );
   glScalef ( 0.3, 0.05, 0.14 );
   glutSolidCube ( 1 );
   glPopMatrix ();
}
```

Por qué así

- La cadena principal **no** hace pop entre niveles: cada nivel hereda del anterior.
- Cada pintar_X() encierra su T · S en un push/pop: la escala de una pieza no llega a sus hijos.

Grados de libertad, teclas y límites



Tecla	Parte	Grado de libertad	Límites	Cursores
1	vehículo	avance: traslación en X	[−2, 2]	← →
2	torreta	giro: rotación en Y	[−170°, 170°]	← →
3	brazo	brazo: rotación en Z	[5°, 80°]	↑ ↓
4	antebrazo	codo: rotación en Z	[−130°, −10°]	↑ ↓
5	pinza	apertura: traslación ±Y de los dedos	[0,04, 0,2]	← →

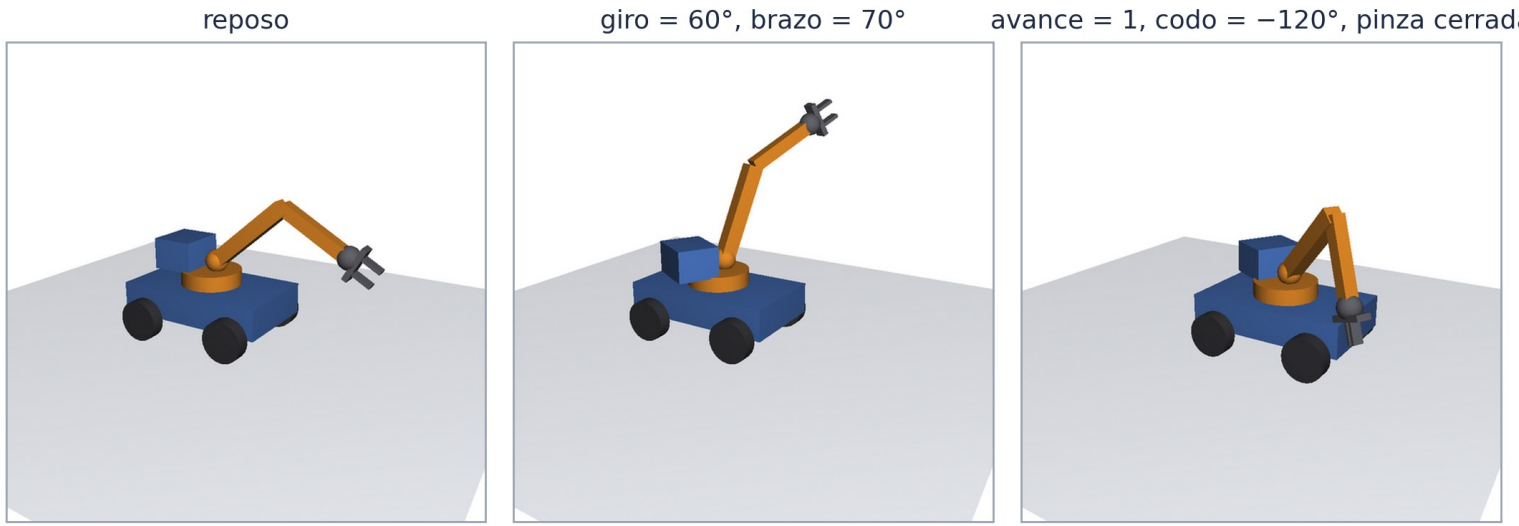
```
void igvRobot::rotarBrazo ( float incr ) {
   brazo = std::max ( 5.0f, std::min ( 80.0f, brazo + incr ) );
}
```

Un método por grado de libertad, como rotarBase(float) del guion de pr2b.

Límites en el modelo

Se comprueban en **un solo sitio** (el método que cambia el grado de libertad): así valen para teclado, ratón y animación.

Mover las partes con el teclado

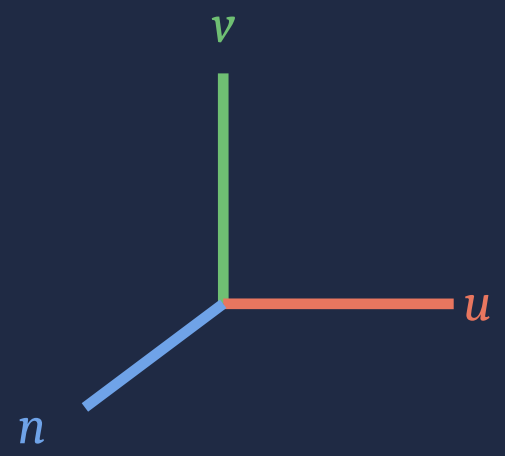



```
// keyboardFunc (modo articulaciones: teclado)
case '1': case '2': case '3': case '4': case '5':
   parte = key - '0';  break;
// specialFunc ( int key, int x, int y )
case GLUT_KEY_LEFT:  robot.mover ( parte, -1 ); break;
case GLUT_KEY_RIGHT: robot.mover ( parte, +1 ); break;
case GLUT_KEY_UP:    robot.mover ( parte, +1 ); break;
case GLUT_KEY_DOWN:  robot.mover ( parte, -1 ); break;
```

mover (parte, sentido)

Llama al método de esa parte con su paso (p. ej. 5° o 0,1) y el signo; cada método aplica sus límites.

3



Selección con el ratón

Buffer de color: cada parte con un color único, como en pr2c.

Qué se pide

pr2c



Clic: elegir la parte

Botón izquierdo sobre una parte del modelo; se identifica con el **buffer de color** (obligatorio este método).

Resaltarla

La parte seleccionada se ve distinta (p. ej. amarilla) y vuelve a su color al elegir otra o pinchar en el fondo.

Arrastrar: moverla

Con el botón pulsado, mover el ratón a izquierda-derecha o arriba-abajo cambia el grado de libertad de esa parte.

Teclado o ratón

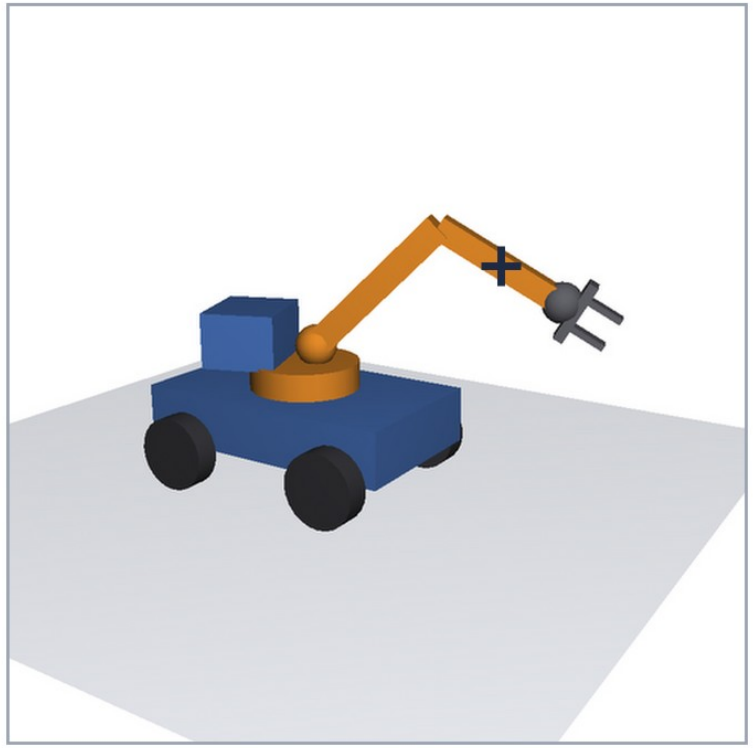
Se mantiene el control por teclado; una **opción de menú** cambia entre el modo teclado y el modo ratón.

Cómo funciona el buffer de color

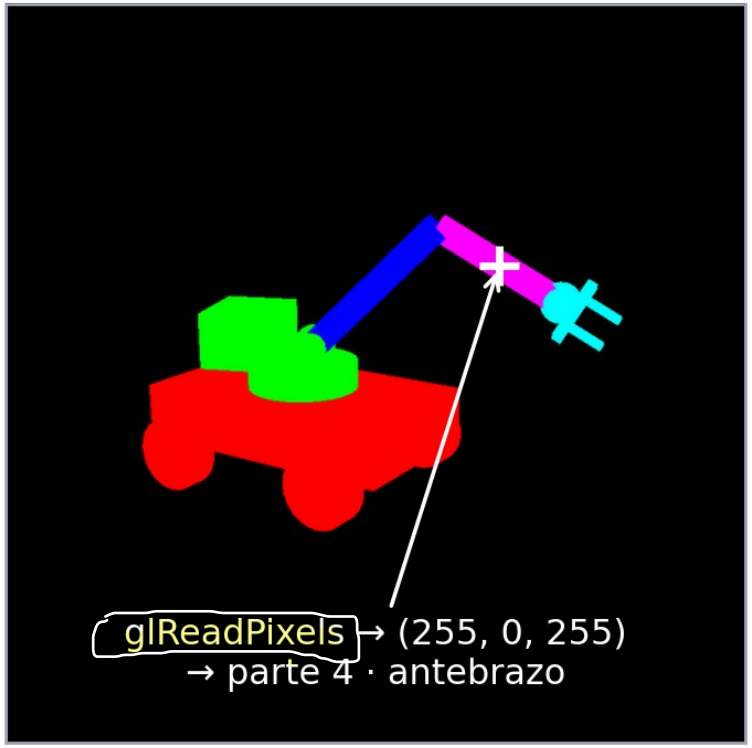
vuelvo a visualizar ya con los colores



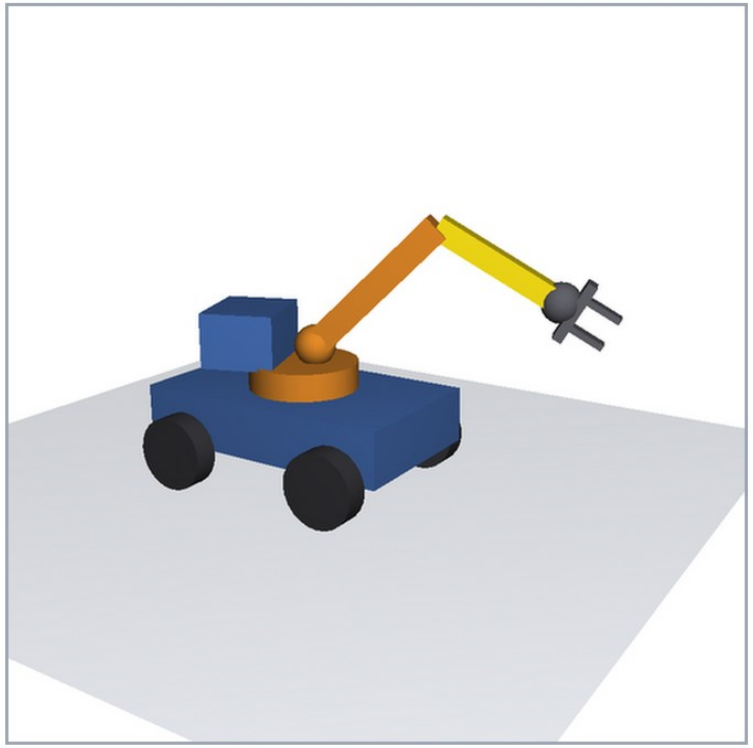
1 · la ve el usuario: hace clic



2 · pasada de selección (no se muestra)



3 · antebrazo seleccionado (amarillo)



1. Clic: se guarda la posición y se pasa a modo selección.

2. Se pinta cada parte de su color, sin luz, y se lee el píxel.

3. Color → parte; se resalta y se vuelve a pintar normal.

La pasada de selección



```
if ( modo == IGV_SELECCIONAR ) {
   glDisable ( GL_LIGHTING );  glDisable ( GL_DITHER );
   glClear ( GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );
   camara.aplicar ();
   escena.visualizar ( true );     // colores de selección
   GLubyte pixel[3];
   glReadPixels ( cursorX, alto_ventana - 1 - cursorY,
                  1, 1, GL_RGB, GL_UNSIGNED_BYTE, pixel );
   parte = robot.parte_de_color ( pixel );  // 0: nada
   robot.set_seleccionada ( parte );
   modo = IGV_VISUALIZAR;
   glEnable ( GL_LIGHTING );  glEnable ( GL_DITHER );
   glutPostRedisplay ();           // pintar el resaltado
}
```

desactivo todo y cuando deje de seleccionar activo todo

true si estamos en seleccionar --> dibujar cada escena de un color

posición del cursor. La ventana tiene un sistema de coords distinto: glut (sup derecha) opengl (abajo izq)

usar GL_MATERIAL

```
// antes de pintar cada parte
void igvRobot::color (bool sel, int id) {
   if ( sel )
      glColor3ubv ( COLOR_SEL[id] );
   else if ( id == seleccionada )
      material ( AMARILLO );
   else
      material ( color_parte[id] );
}
```

Lecciones de pr2c: fila **alto − 1 − y** (la y de GLUT crece hacia abajo); **no** pintar en amarillo durante la pasada de selección; reactivar GL_DITHER; y **glutPostRedisplay** al terminar, para que el resaltado se vea sin esperar a mover el ratón.

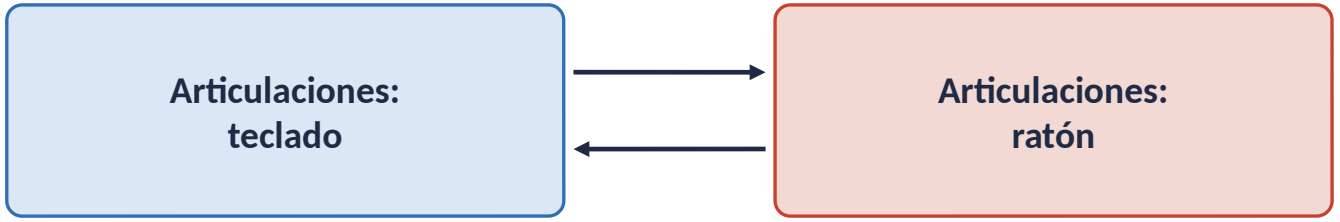
Arrastrar para mover y cambiar de modo



```
void igvInterfaz::motionFunc ( GLint x, GLint y ) {
   if ( boton_retenido && parte > 0 ) {
      int dx = x - cursorX;
      int dy = cursorY - y;          // y de GLUT: hacia abajo
      robot.arrastrar ( parte, dx, dy );  // con límites
      cursorX = x;  cursorY = y;
      glutPostRedisplay ();
   }
}
```

arrastrar (parte, dx, dy)

- Partes con giro horizontal (vehículo, torreta, pinza): **dx**.
- Brazo y antebrazo: **dy** (subir o bajar).
- Un factor de escala (p. ej. 0,5° por píxel) suaviza el movimiento.

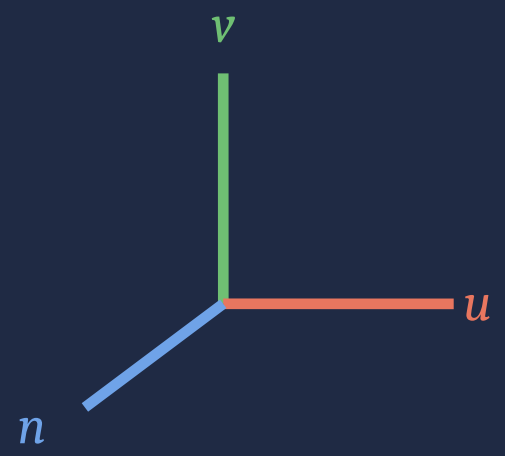


opción de menú «Interacción»

El modo es un estado más

Un enum en igvInterfaz. keyboardFunc, specialFunc, mouseFunc y motionFunc lo consultan antes de actuar.

4

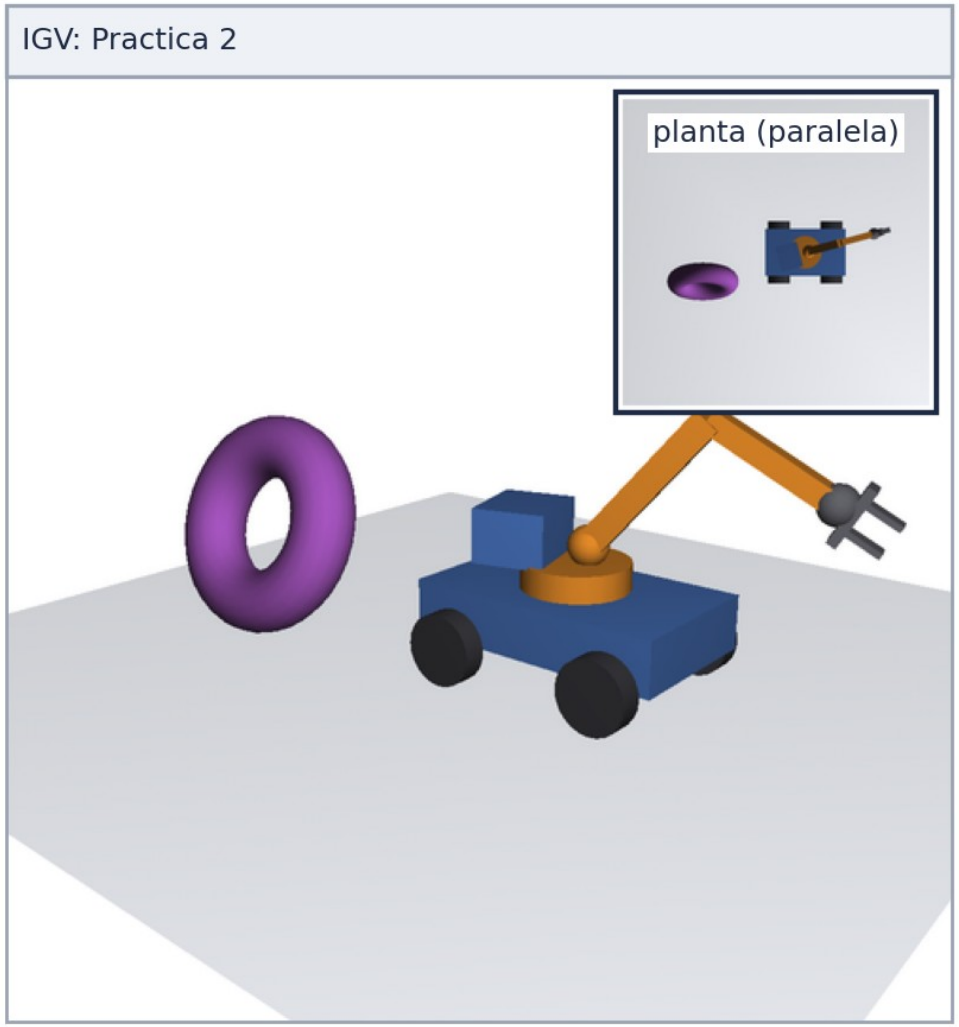


Integración de la Práctica 1

Transformaciones, cámara y viewports sobre la escena con los dos modelos.

Qué se integra

NO CAE NADA DE LA PRIMERA PRÁCTICA



- **Los dos modelos a la vez**: la malla paramétrica y el modelo articulado, visibles y operables en la misma escena.
- **Transformaciones geométricas** (traslación, rotación y escalado) sobre cada modelo **completo**, como los objetos de la Práctica 1.
- **Movimientos de cámara**: órbita, cabeceo, giro, recorte, zoom y tipo de proyección.
- **Viewports**: la vista principal y la vista de planta.

Las transformaciones de la P1 mueven el modelo **entero**; los grados de libertad mueven **sus partes**. Son cosas distintas y conviven.

Una escena, dos modelos, cada uno con su estado



```
void igvEscena3D::visualizar ( bool sel ) {
   if ( ejes && !sel ) pintar_ejes ();
   glPushMatrix ();                        // la malla
   aplicar ( trans[MALLA] );               // T · S · R (P1)
   if ( sel ) glColor3ub ( 0, 0, 0 );      // tapa, no se elige
   malla->visualizar ();
   glPopMatrix ();
   glPushMatrix ();                        // el robot
   aplicar ( trans[ROBOT] );
   robot.visualizar ( sel );               // sus GDL dentro
   glPopMatrix ();
}
```

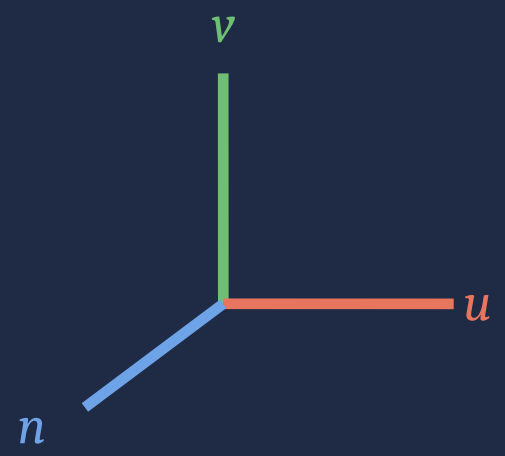
trans[i], como en la P1

- Traslación, ángulos y escala acumulados **por separado** para cada modelo.
- Se aplican en el orden $M = T{\cdot}S{\cdot}R$, dentro de su push/pop.

En la pasada de selección la malla se pinta del color del fondo: así **tapa** lo que tenga detrás y no se selecciona una parte oculta.

Con dos viewports, la selección debe hacerse con la **cámara y el viewport** de la vista donde se ha hecho clic (o ignorar los clics en el recuadro).

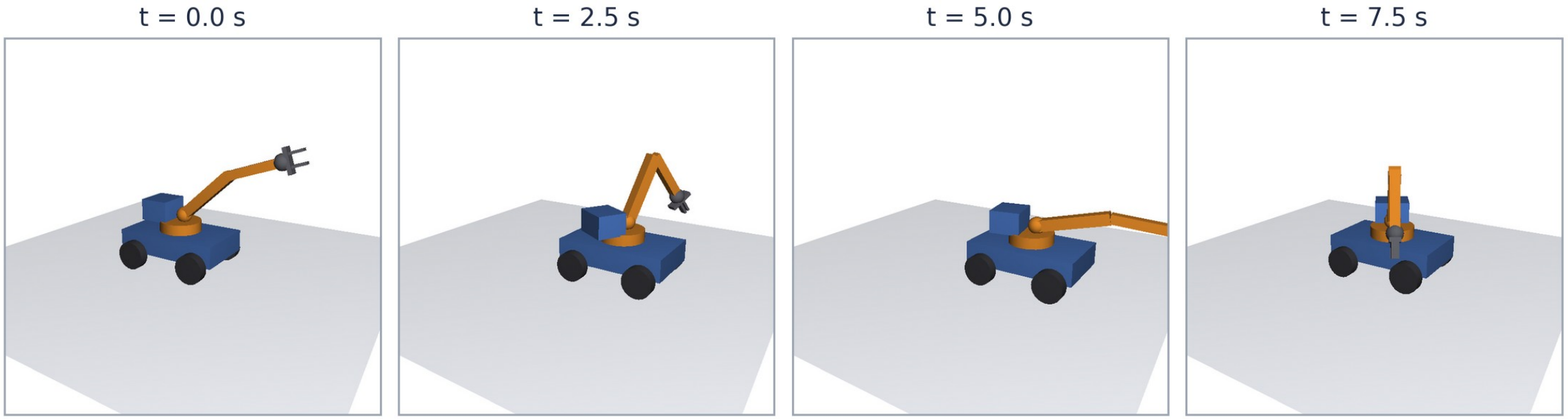
5



Animación automática

El modelo se mueve solo y la cámara orbita, activados desde el menú.

Animación del modelo



Toda la escena

Se anima el modelo articulado con **sus grados de libertad** y, si se quiere, también la malla.

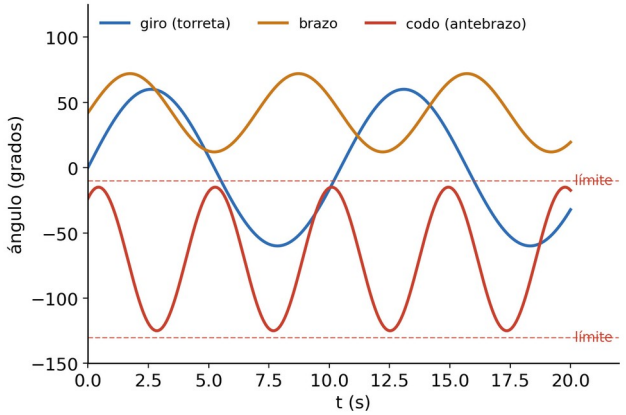
Realista

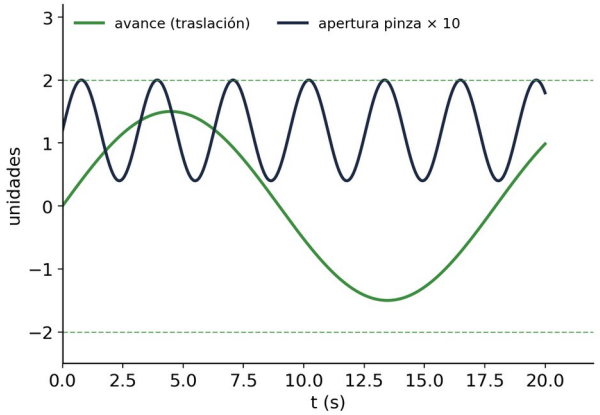
Movimientos suaves y dentro de los **límites**: ninguna pose imposible.

Por menú

Una opción activa y detiene la animación; el usuario puede seguir usando el resto.

idleFunc: tiempo real, límites y CPU






```
void igvRobot::animar ( float dt ) {
   fase  += dt;
   avance = 1.5 * sin ( 0.35 * fase );
   giro   = 60 * sin ( 0.6 * fase );
   brazo  = 42 + 30 * sin ( 0.9 * fase );
   codo   = limitar ( -70 + 55 * sin ( 1.3 * fase + 1 ),
                      -130, -10 );
   pinza  = 0.12 + 0.08 * sin ( 2.0 * fase );
}
```

animar objetos + cámara

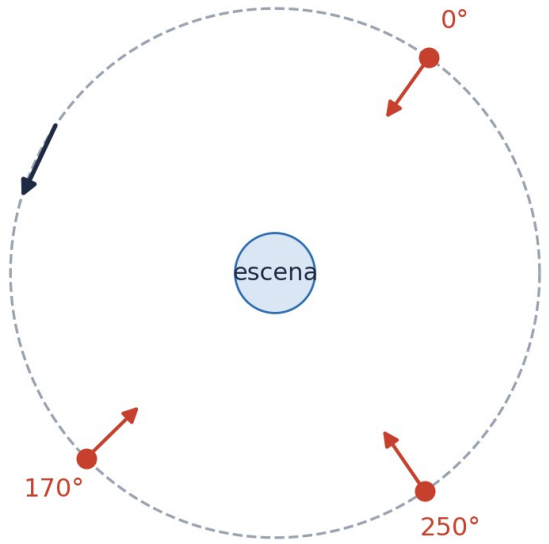
función que limita los grado de libertad

```
void igvInterfaz::idleFunc () {
   int t = glutGet ( GLUT_ELAPSED_TIME );    // ms
   float dt = ( t - _instancia->t_prev ) / 1000.0f;
   _instancia->t_prev = t;
   if ( _instancia->animar_modelo )
      _instancia->escena.animar ( dt );
   if ( _instancia->orbitar_camara )
      _instancia->camara.orbita ( 20 * dt ); // 20°/s
   glutPostRedisplay ();
}
// al cambiar las opciones en el menú:
glutIdleFunc ( animar_modelo || orbitar_camara
               ? idleFunc : nullptr );
```

booleano

Con **dt** la velocidad no depende del ordenador; sin idleFunc registrada no se gasta CPU en reposo.

Órbita automática de la cámara



vista desde arriba: P_0 gira alrededor del eje Y; r fijo en la escena



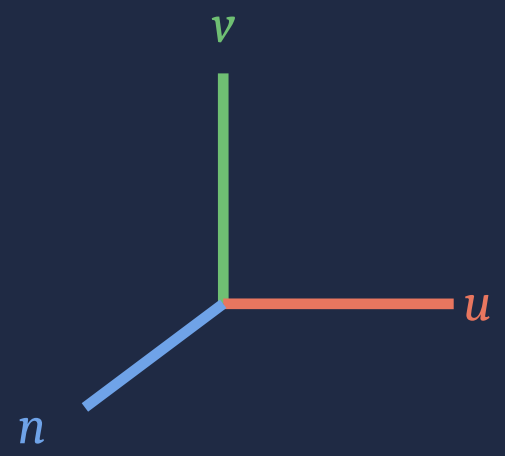
Reutilizar la órbita de la P1

En cada paso, $P_0 \leftarrow r + R_y(\alpha \cdot dt) \cdot (P_0 - r)$: el mismo método orbita() de igvCamara, con un ángulo proporcional al tiempo.

Independiente de la animación

Se activa por menú. Puede ir a la vez que la animación del modelo; al pararla, la cámara se queda donde está.

6



Experiencia de usuario

Menús, teclado y ratón bien organizados: es un aspecto que se valora.

Un menú organizado en submenús



menú principal (botón derecho)

1 Objeto: sombreado ▸
2 Interacción ▸
3 Animación ▸
4 Cámara y vistas ▸
Ayuda (teclas)
Salir

1 Objeto: sombreado
Plano (GL_FLAT)
Interpolado (GL_SMOOTH)

2 Interacción
Transformar modelos (P1)
Articulaciones: teclado
Articulaciones: ratón
Mover la cámara

3 Animación
Animar modelo: sí / no
Orbitar cámara: sí / no

4 Cámara y vistas
Perspectiva / paralela
Vista de planta: sí / no
Reiniciar cámara

`glutCreateMenu · glutAddMenuEntry · glutAddSubMenu · glutAttachMenu ( GLUT_RIGHT_BUTTON )`

```
int sub1 = glutCreateMenu ( menuHandle );
glutAddMenuEntry ( "Plano", M_PLANO );
glutAddMenuEntry ( "Gouraud", M_GOURAUD );
int sub2 = glutCreateMenu ( menuHandle );
glutAddMenuEntry ( "Teclado", M_TECLADO );
glutAddMenuEntry ( "Raton", M_RATON );
// … sub3 y sub4 igual
glutCreateMenu ( menuHandle );  // raíz
glutAddSubMenu ( "Sombreado", sub1 );
glutAddSubMenu ( "Interaccion", sub2 );
glutAddMenuEntry ( "Salir", M_SALIR );
glutAttachMenu ( GLUT_RIGHT_BUTTON );
```

Agrupar

Un submenú por tema. Las opciones que alternan dicen su estado (sí / no).

Sin tildes

Mejor sin tildes ni ñ en las etiquetas: según el sistema se ven mal.

Un mapa de teclas sin conflictos



Tecla	Transformar modelos (P1)	Articulaciones: teclado	Cámara
1, 2, …	elegir modelo (1 malla, 2 robot)	elegir parte (1 … 5)	—
← →	traslación en X	grado de libertad de la parte	órbita
↑ ↓	traslación en Z	grado de libertad de la parte	cabeceo
x X · y Y · z Z	rotaciones	—	y / Y: giro
s S · u U	escala · traslación en Y	—	—
f F · b B · + − · p P	recorte, zoom y proyección, en todos los modos		

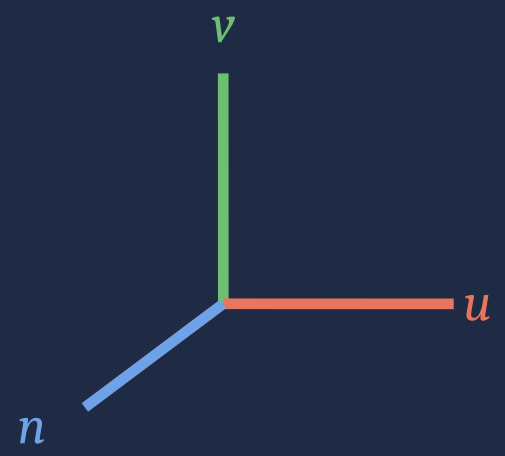
El modo, siempre a la vista

Por ejemplo en el título de la ventana con glutSetWindowTitle: «Modo: articulaciones (teclado) · parte: brazo».

Ayuda

Una opción de menú (o la tecla h) que muestre en consola las teclas y el uso del ratón del modo activo.

7



Entrega

Qué hay que entregar y cómo comprobar que está todo.

Qué entregar



Código C++ autodocumentado

Con las clases proporcionadas. Todos los .cpp y .h, aunque no se hayan modificado, y los ficheros de configuración (.cmake, .py, .yml, .txt). Nada de .o ni ejecutables.

Vídeo explicativo

La aplicación en funcionamiento, enlazado desde la memoria y descargable (p. ej. Google Drive), con permisos para cualquier docente.

Implementación y funcionamiento

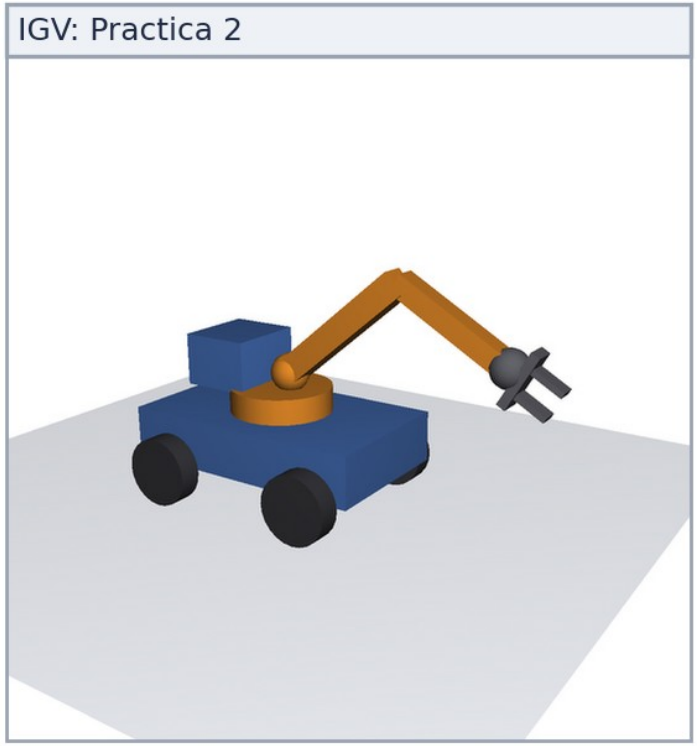
Detalles fundamentales de lo realizado y cómo se usa: teclas, opciones de menú y movimientos del ratón.

El modelo jerárquico

El grafo de escena con las transformaciones de cada nodo, indicando **dónde** están las que dan los grados de libertad.

Normas de envío

- Sube los ficheros **solo uno** de los dos miembros de la pareja.
- Un **.zip** con los nombres de los dos miembros en el nombre del fichero.
- El proyecto debe poder compilarse y ejecutarse **sin añadir ficheros**: todos los .cpp y .h y la configuración (.cmake, .py, .yml, .txt).
- En la explicación, los **nombres** de la pareja y el **enlace al vídeo**, comprobando los permisos de descarga.
- Entregar en la **fecha fijada** a principio de curso: es requisito para evaluarla y para presentarse al **examen práctico**.



Lista de comprobación para el vídeo



- [x] El objeto paramétrico propio, con arrays de vértices y de normales, y el cambio plano / interpolado desde el menú.
- [x] El grafo de escena: al menos 4 niveles, 12 nodos y una estructura no lineal.
- [x] Cada grado de libertad (al menos 4, alguno de traslación) con 1, 2, 3… y los cursores, sin pasar de los límites.
- [x] Selección con el ratón por buffer de color: la parte se resalta y se mueve arrastrando.
- [x] El cambio entre modo teclado y modo ratón desde el menú.
- [x] Las transformaciones de la P1 sobre los dos modelos, la cámara y los dos viewports.
- [x] La animación automática del modelo y la órbita de la cámara, activadas y paradas desde el menú.
- [x] Los menús, las teclas y el uso del ratón, explicados.

Resumen



1 Objeto paramétrico

Malla propia con arrays de vértices y normales (por fórmula o por caras) y glDrawElements; plano o Gouraud por menú.

2 Modelo articulado

Árbol con push/pop por rama; ≥ 4 niveles, ≥ 12 nodos y ≥ 4 GDL con límites en el modelo.

3 Selección

Buffer de color, parte resaltada y arrastre; teclado o ratón según el modo elegido en el menú.

4 Práctica 1

T · S · R por modelo completo, cámara y viewports sobre la escena con los dos modelos.

5 Animación

idleFunc con tiempo real, dentro de los límites, y órbita automática de la cámara.

6 Usuario

Submenús, un mapa de teclas sin conflictos y el modo siempre visible.